DEPLOY & SHARE · V2.3.0 · CODE TAB + COWORK

# Claude Code Toolkit — Deploy & Share

Everything you need to install this toolkit on your own machines and share it with others — across **both surfaces** of the Claude Desktop app: the **Code tab** (via a plugin) and **Cowork** (via skills). No terminal required.



## The two surfaces

The Claude Desktop app has two places you build with Claude, and the toolkit ships for each:

| SURFACE | WHAT IT IS | HOW THE TOOLKIT SHIPS | YOU USE |
|---|---|---|---|
| **Code tab** | the software-development tab (slash commands, panes) | a **plugin** (marketplace install) | `/plan` `/build-all` `/review` … |
| **Cowork** | the agent-mode tab for longer autonomous work | **skills** (`.skill` files) | `bounded-build`, `code-review` |

> Same ideas, different packaging: the Code tab uses slash commands from a plugin; Cowork uses skills. Both keep Claude sharp on long work by **running each stage in its own subagent** so the main context stays small.

## What's included

### Code-tab plugin (`claude-code-toolkit`)

- **Bounded-build:** `/plan /stage /handoff /verify /reset /ship /trim`
- **Auto-advance:** `/build-all` (checkpoint) & `/build-all-auto` (full-auto)
- **In-session review:** `/review` + `review-agent`
- **Subagents:** planner, verifier, researcher, debugger, security-reviewer, network-infra, web-cloudflare, stage-runner
- **Safety hooks:** destructive-command guard, secret-scan, session primer (auto-loads the operating rules)
- **Output style:** terse-engineer

### Cowork skills (`cowork/`)

- **bounded-build** — plan to disk, run each stage in a subagent; checkpoint or full-auto; halts on a failed exit criterion.
- **code-review** — reviews your working `git diff`; emits `APPROVE / REQUEST_CHANGES / REJECT` with `file:line` findings.

Delivered as installable `.skill` files plus editable source folders.

## Deploy on your own machines

### A · Code tab (the plugin) — no terminal

1. Open the Claude Desktop app → `Code` tab → `+` → **Plugins** → **Add plugin**.

2. Add the marketplace `justinw916-sketch/SuperClaudePublic`, then install **claude-code-toolkit**.
3. **Fully quit and reopen the app** (tray/menu-bar → Quit). Plugin changes only apply after a full restart.
4. Verify: type `/` → you should see `/plan`, `/build-all`, `/review`, etc. The operating rules load automatically (SessionStart hook) — nothing to paste.

> **Windows:** the first Code-tab launch needs Git for Windows installed; restart the app after installing.

### B · Cowork (the skills)

1. Get the two skill packages from the repo's `cowork/` folder: `bounded-build.skill` and `code-review.skill`.
2. In Cowork, open each `.skill` file and click **Save skill**. (You only need to save each once — if it asks to "replace", that just means it's already saved.)
3. Use them by describing a substantial build (`bounded-build` triggers) or asking to "review my changes" (`code-review` triggers).

### C · (Optional) personal identity

The operating rules load automatically via the plugin, so this is optional. To personalize, paste `reference/CLAUDE.md` into your `~/.claude/CLAUDE.md` and fill in your name/stack.

## Share with a friend

Everything lives in the public repo **github.com/justinw916-sketch/SuperClaudePublic** (MIT-licensed, no personal data in files or git history).

### For their Code tab

They do the same 4 steps as above: `+` → Plugins → Add plugin → add `justinw916-sketch/SuperClaudePublic` → install → restart.

### For their Cowork

Send them the two `cowork/*.skill` files (or point them at the repo's `cowork/` folder). They open each and click **Save skill**.

> Rendered docs are public at justinw916-sketch.github.io/SuperClaudePublic — hand your friend that link and they can read everything without cloning.

## Update to the latest version

Plugins don't auto-update. When a new version ships (this is **v2.3.0**):

### Code tab

1. `+` → Plugins → **claude-code-toolkit** → **Update** if shown.
2. No update button? **Remove** the marketplace, **re-add** it, then **reinstall** — re-adding forces a fresh pull.
3. **Fully restart** the app.

CLI equivalent (if used):

```
claude plugin marketplace update superclaude-public
claude plugin update claude-code-toolkit@superclaude-public
# then restart
```

### Cowork

Re-open the updated `cowork/*.skill` file and click **Save skill** → **Upload and replace**. That swaps in the new version.

> The marketplace **cache** is what needs refreshing — just clicking "install" again can reuse a stale copy. Re-adding the marketplace (or `marketplace update`) guarantees the latest.

## The build workflow

Plan once, then let stages run — each in its own fresh context. Does each command stop or keep going?

| COMMAND | WHAT IT DOES | STOPS FOR YOU? |
|---|---|---|
| `/plan` | writes the plan; builds nothing | **Stops** — review, then `/stage` or `/build-all` |
| `/stage N` | runs one stage, then stops | **Stops after each stage** (manual) |
| `/build-all` | runs all stages, narrating each | Auto-continues; checkpoints don't wait; halts only on failure |
| `/build-all-auto` | runs all stages silently | Auto-continues to the end; halts only on failure |
| `/review` | reviews your diff | **Stops** with a verdict |

| | | |
|---|---|---|
| `/ship` | pre-commit checklist, then commit | **Waits** for your go-ahead |

> **Checkpoints don't need input** — they're status messages; the run continues on its own. The only workflow-level stops are a **failed exit criterion** or an explicit pause request. For a truly hands-off run, set the app to **Auto** permission mode so it doesn't pause for tool-approval prompts.

In **Cowork**, the `bounded-build` skill runs the same loop — invoke it by describing the build, and add "run all stages" for full-auto.

## Why it saves tokens and keeps intelligence

In a long session, every message re-processes the whole accumulated context — cost climbs and quality degrades as the window fills ("context rot"). Running each stage in a **subagent** fixes both: the subagent spends its tokens in its own fresh context and returns only a short summary, so your main thread stays small, cheap, and in the high-quality band.

| | ONE LONG SESSION | SUBAGENT PER STAGE |
|---|---|---|
| Main context by late stages | large & climbing | small & flat |
| Re-processing old bulk each turn | yes | no |
| Quality late in the build | degrades | preserved |

> Honest framing: the subagents still spend tokens doing the work — the win is that the **main thread stays cheap** and **quality is preserved**. The bigger the job, the larger the gap. For tiny one-shot tasks, skip it — the overhead isn't worth it.

## Troubleshooting

- **A new command doesn't appear** — fully quit and reopen the app (not just the tab).

- `claude plugin list` **says "disabled"** — that CLI status is unreliable; the authoritative signal is `enabledPlugins: true` in `~/.claude/settings.json`. Confirm in the app by typing `/`, or via `+` → Plugins.
- **"Replace skill?" when saving a Cowork skill** — expected if it's already saved; safe to Cancel or replace. You only need to save each `.skill` once.
- `/agents`, `/config`, `/permissions`, `/doctor` **"isn't available"** — those are terminal-only dialogs; manage them via **Customize** in the sidebar and Settings.
- **Windows, first Code-tab launch fails** — install Git for Windows, then restart the app.
- **Build stops mid-run** — that's the gate: a stage failed its exit criterion. Fix it, then re-run `/build-all` to resume.

---

**Links.** Repo: SuperClaudePublic · Rendered docs: GitHub Pages · Code-tab guide: using-in-claude-code-desktop.html · Scheduled tasks: scheduled-task-recipes.html.

**Sources.** Claude Code Desktop app · Plugins · Hooks. Context-rot framing per Chroma "Context Rot" (2025) and Anthropic "Effective context engineering."

Claude Code Toolkit — Deploy & Share · v2.3.0 · covers Code tab + Cowork.